

AI Engineering Case Study

Ticket Intelligence — build a retrieval-augmented system that turns a year of operational tickets into a briefing a manager can act on, and prove it does not make things up.

The scenario

A city's complaints line produces about 26,000 tickets a year. An operations manager has to decide where to put staff next quarter. They cannot read 26,000 tickets, and the existing dashboard has not helped — it shows what is **big**, not what has **changed**.

You are building the AI system that closes that gap: it reads everything, works out what is worth saying, and answers questions about it with evidence attached.

The objective

Turn 26,000 tickets into at most 50 findings, then serve them through a grounded question-answering endpoint.

Every finding names a timeframe, cites at least three real ticket IDs, and carries a confidence you can explain.

That is a 500:1 compression. The three requirements — timeframe, citations, confidence — exist to stop the system producing fluent text nobody can verify, which is the characteristic failure of this class of project.

The architecture you are asked to build

```
26,000 tickets
  ↓ LAYER 1 — pattern detection (statistics + LLM where it earns its place)
~50 findings, each with citations and a confidence
  ↓ LAYER 2 — embed the findings, index them
a vector index of 50 items
  ↓ LAYER 3 — retrieve, rank, answer with citations
grounded answer + the tickets that back it
```

Layer 1 — detection

Find what changed. Counting tickets per week per category and testing for a step change is arithmetic and costs nothing; describing *what the tickets actually say* is a language problem and needs a model. **Use each where it belongs** and say in your README which findings came from which.

Layer 2 — the index

Embed the **findings**, not the tickets. Fifty vectors, not twenty-six thousand. Any embedding model, any store — a numpy array is a perfectly good vector database at this size, and saying so in your README is a better answer than installing one you do not need.

Layer 3 — grounded answering

Retrieve relevant findings, rank them, and have the model answer **only from what was retrieved**. Every claim in the answer traces to a finding, and every finding traces to ticket IDs. Similarity alone is rarely the right ranking — consider blending it with recency and importance, and be ready to defend

your weights.

The data

data/tickets.csv — 25,921 rows, 4.8 MB, calendar year 2025.

Column	Meaning
ticket_id	Unique identifier. Your citations must use these.
created_date	When the complaint was raised. Your clock.
closed_date	When it was resolved. Empty if still open.
agency	Which department handled it.
complaint_type	Category, e.g. "Noise - Residential".
descriptor	Sub-category, e.g. "Loud Music/Party".
borough	Where it happened.
status	Closed, Open, In Progress or Pending.
resolution_description	Free text describing what was done. Empty if not closed.

Synthetic, but modelled on a real municipal 311 export, so the shape and the messiness are realistic. **There are genuine patterns in it — and at least one thing that looks like a pattern and is not.**

What to submit

#	Deliverable	What it is
1	mine.py	python mine.py --input data/tickets.csv --out findings.json — produces at most 50 findings
2	A service	POST /ask — takes a question, returns a grounded answer with the findings and ticket IDs behind it
3	evaluate.py	Runs evaluation/questions.json against your service and prints a score you chose and can defend
4	README.md	One page. Must answer: what did your system refuse to emit, and why?

Validation and evaluation

This is a third of the exercise, not a closing paragraph. A system that produces findings nobody has checked is not finished.

A · Validating the findings — is the system RIGHT?

Every finding makes a claim about the data. Each one should be reproducible from the CSV without going through your pipeline.

- **Re-derive it.** A finding saying “noise rose 3x from June” should be checkable with a one-line group-by on the raw file. Write those checks down; a reviewer will run them.
- **Resolve every citation.** Each cited ticket ID must (i) exist in the input, (ii) be of the complaint type the finding is about, and (iii) fall inside the stated window. All three are automatable — automate them.
- **Use a negative control.** Pick a slice with no real pattern and confirm your system emits nothing for it. If it always finds something, it is not detecting — it is describing.

B · Evaluating retrieval — does it find the RIGHT findings?

For each of the five questions, label by hand which of your findings *should* come back. That labelled set is your ground truth. Then measure:

Metric	Question it answers	How
Precision@k	Of what came back, how much was relevant?	relevant retrieved ÷ k
Recall@k	Of what should have come back, how much did?	relevant retrieved ÷ all relevant
MRR	How high up was the first good one?	1 ÷ rank of first relevant result

Five questions is a small test set and you should say so. Adding your own questions with labelled answers is one of the strongest things you can do with spare time.

C · Evaluating the answer — is it GROUNDED?

Retrieval can be perfect and the answer still wrong, because the model added something that was not there. Measure that separately.

- **Faithfulness.** Split the answer into claims. What share of them is supported by a retrieved finding? Anything unsupported is a hallucination, however plausible it sounds.
- **Citation accuracy.** Of the ticket IDs in the answer, what share genuinely back the statement they are attached to?
- **Abstention.** On the question with no good answer, does it decline? This is pass or fail, not a percentage — and it is the single most informative number in your report.

D · What to report

`python evaluate.py` should print something a reviewer can read in ten seconds:

Q	Retrieved	P@3	Recall	Faithful	Citations OK	Verdict
1	3	1.00	1.00	1.00	3/3	pass
2	3	0.67	1.00	1.00	4/4	pass
3	3	0.33	0.50	1.00	2/2	partial
4	2	1.00	1.00	1.00	3/3	pass
5	0	—	—	—	—	abstained — correct

Plus, for the whole run: **tokens consumed, wall-clock time, and how many findings were refused.** An estimate with your working shown is fine.

An LLM judge is allowed for faithfulness, and is a reasonable choice at this scale — but say so, show the prompt, and spot-check a few of its verdicts by hand. A judge nobody audited is just another unverified model.

Engineering constraints

1. **At least two findings must be produced with no model call at all.** Counting and a step-change test is arithmetic. Running 26,000 tickets through an LLM answers a different question and spends real money doing it — we want to see you know the difference.
2. **Refusal must be explicit.** Write down the rules that reject a finding, and report how many you discarded. A run that rejected nothing has not understood the problem.
3. **Every citation must resolve.** Each ticket ID you cite must exist in the input and support the claim. We will check a sample.
4. **Running it twice must not double the output.** Mine January, then mine January and February — you must not end up with two copies of one finding. Decide what makes two findings “the same” and write it down.

5. **Report your cost.** Tokens consumed and wall-clock time for a full run. An estimate with your working shown is fine.

How this is assessed

Signal	What we look for
Grounding	Does every answer trace to findings, and every finding to real tickets?
Restraint	Did anything get refused — and can you explain why that was right?
Compression	26,000 → ~50. Emitting 5,000 findings misses the point entirely.
Method choice	Statistics where statistics suffice; the model where language is the problem.
Calibration	Does confidence mean something, or is it always 0.9?
Evaluation	Did you measure your own system, and is the measure sensible?
Honesty	Does it say “I don’t know” when it should?

Scope, by the time you have

Time	Aim for
2 days	Counting, one change detector, embed the findings, /ask, the five questions running.
1 week	Add a second detection method, LLM summarisation of finding text, the refusal rules, the merge rule.
2 weeks	Add clustering over resolution_description, a real evaluation score, and a rule that drops stale findings.

We would rather see a small system you can defend than a large one you cannot. If you run out of time, write down what you would have done next and why. That is worth more than half-finished code.

Submit a git repository or a zip: your code, findings.json from a full run, your evaluation output, and the README. Include how to run it. Any language, any framework, any model.